

Response to White House and OSTP release of RFI concerning the Development of an Artificial Intelligence (AI) Action Plan

From:

Peter Galicki (individual inventor)

March 10, 2025

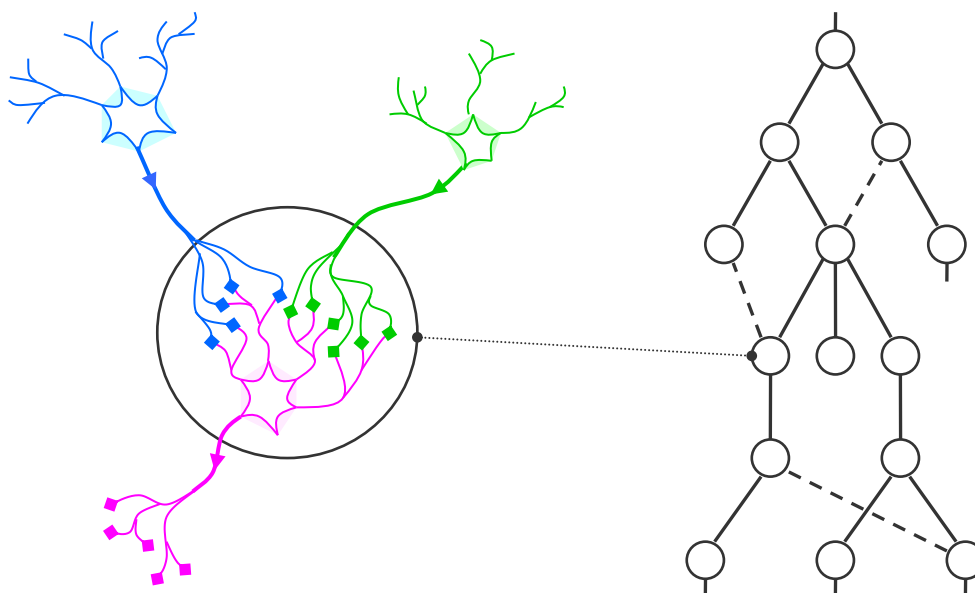
This document is approved for public dissemination. The document contains no business-proprietary or confidential information. Document contents may be reused by the government in developing the AI Action Plan and associated documents without attribution.

Introduction:

This response suggests concrete policy actions addressing three AI policy topics:

- Hardware and chips
- Model development
- Energy consumption and efficiency

The AI policy action suggestions listed below are based on author's privately sponsored development of computer hardware and programming models for AI applications. The current status and future plans for this work are listed on the last page of this document. The referred hardware and programming models can be best described as a computer platform that mimic Neural Messaging. They represent a major shift from legacy Load-Store computer architectures to message-based computer hardware that works like a human brain.



Recommended AI policy actions for hardware and chips

- **Replace Load-Store computer architectures with Neural Messaging architecture.** Load-Store architectures used with legacy CPUs are inadequate for systems with thousands of processors because they concentrate data traffic (as implied by the word “Central” in CPU). In massively parallel systems this leads to data congestion and lockups. Data Flow architectures such as Neural Messaging concurrently flood the entire computer with thousands of data messages running across thousands of processors to trigger, exchange and aggregate computing results. This method inherently spreads out data traffic across the entire system to reduce congestion and simplify data transfers.
- **Apply Parallel Processing inside processing threads.** In addition to parallelizing multiple threads, this will spread out single thread processing amongst multiple processors, further increasing performance. Use synchronization hardware to account for some operands not yet being available at the time when requested.
- **Use common instructions that work on both scalar and vector operands.** With multiple processors available to work on a single thread, there is no need for separate scalar and vector instructions. Both scalar and vector functions can be performed on common adders, multipliers, etc. using variants of common instructions. This reduces the size of processors and the complexity of the instruction set.
- **De-couple instruction execution logic from instruction control logic.** This will make it easy to add new ALUs to Data Processing Units.
- **Package data and instructions into packets** that flow freely between subsystems on multi-dimensional “data highways”, instead of using legacy buses to explicitly drive data. This increases overall data flow bandwidth, automates arbitration for shared resources and reduces and spreads out data congestion.
- **Remove inefficient read cycles from processing flows.** Read cycles are inherently serial in nature and are not compatible with parallel computing flows. Instead, pass the read addresses through memories to collect read operands on the way to the Data Processing Units. This eliminates round-trips that would otherwise slow the execution pipelines.
- **Apply active and passive controls to regulate data traffic** between thousands of components in the system. This reduces data congestion and removes the possibility of occurrence of a catastrophic gridlock condition.

- **Distribute data processing to thousands of small processors.** Replace large legacy CPUs with many smaller efficient TPUs (Thread Processing Units). Thousands of TPUs, along with small memories evenly distributed within the system, can process more threads in parallel than a smaller number of large CPUs. Visualize a self-propagating processing wave sweeping the multi-dimensional computer system to autonomously process and aggregate partial results into a final results. No active control required to assemble the results as is the case with legacy CPUs.
- **Reduce dependence on LIFOs for propagating program context.** Stacks and other structures using Last-In-First-Out memories have an inherently serial nature that inhibits parallel execution. Use parallel means to propagate context at program boundaries.
- **Automate in hardware the assignments of computing tasks to processors.** Software driven controllers cannot efficiently make task-processor assignments when thousands of processors are involved. Such central planning is not compatible with free-flowing data and program streams that service distributed computing resources of AI systems.
- **Structure instruction fetch mechanisms to efficiently deliver small programs** to thousands of processors working concurrently.
- **Reduce addressing mode complexity.** Small processors working on small snippets of code operating within small memory ranges typically assigned to threads do not need multitudes addressing mode choices. Complex addressing modes are a relic of legacy CPUs. Direct mode addressing is all that is needed for swarms of small processors working together.
- **Employ new types of instructions to efficiently spawn neural messages.** In Data Flow architectures such as Neural Messaging, messages are the dominant form of sharing data between thousands of processors. New types of instructions are needed to process these messages efficiently, such as using just one instruction to assemble and cast multiple messages to multiple recipients suspended in many locations of the communications fabric.

Recommended AI policy actions for model development

- Model the AI computing tasks with **Neural Messages** traversing arrays of **Neural Objects** interconnected with network of **Neural Links**.
- **Neural Objects** are a collection of related properties and behaviors that can be independently processed and duplicated to rapidly build many classes of objects. These properties exist in the form of static data, dynamic data, transit data and program code. Neural Messages passing through Neural Objects apply the embedded transit data to Object data as instructed by Object's property code. Some of the Object's property data classified as Navigational Data is used to direct the passing of Neural Messages to other Objects.
- **Neural Messages** pass through and between Neural Objects to process Objects' properties. The contents of Neural Messages can represent more than one type of transit data, each type being associated with a traverse direction a Message may be taking. For example, forward transit data may represent one direction as described by navigation rules. Return transit data could be associated with another direction of travel. Each Neural Message can trigger processing of associated properties inside the Object and the navigational properties for links to other Objects. The navigation code can propagate Neural Messages to other Objects or block them from further propagation. Code processing the properties can result in updates to the state of the active property and that of related properties. Based on process action potential, a Neural Message associated with one property may also spawn Neural Messages associated with related properties. This is analogous to how Synapses open and close channels to manage propagation of messages between neurons based on action results coming from Dendrites in the human nervous system.
- **Neural Links** connect Neural Objects for the purpose of conveying Neural Messages through and between Neural Objects. Links can be associated with a direction of traverse reflecting the hierarchy in which the Neural Objects are arranged. For example, there could be "up" links connecting to parent objects or "down" links connecting to daughter objects. Object properties may be processed differently depending on the type of Neural Link they arrive through into the Neural Object. Neural Links can also be re-configured based on results of Message processing. For example, one end of a Link may be disconnected from a Neural Object to be connected to another Object, in effect changing the flow of future Neural Messages between Objects. This enables the Neural

Computer to effectively re-wire itself as it learns and adapts to new experiences reflected in Object's properties.

- **Use Bottom-Up task processing models** instead of the established top-down models. Top-down approaches rely on Central Control coming from top levels of program hierarchy. The complexity and rigidity of rules coming from the top make Central Control incompatible with massively parallel systems that flow more efficiently with the fewest restrictions. Massive numbers of self-propagating and self-regulating Neural Messages require no Central Control as they autonomously flow through thousands of Neural Objects executing the many small core property programs. This bottom-up approach bypasses all traditional controls associated with top-down approaches to directly zero-in on core inner loops to process incremental results that self-assemble into the final results. Bottom-up models are analogous to Calculus in that they divide the processing task into small chunks before aggregating the incremental results into full solutions.
- **Use common Templates to process properties inside Neural Objects.** A common flow may be used as a sequential template for sourcing code to process Object properties with associated transit data embedded in the arriving Neural Messages. First, the transit data may be transformed by the dynamic data to bring it to the same processing space as the Object's static data. Next, the static data could be applied to the transformed transit data to find the desired relations. Based on the computed relations, actions may be taken in the property dynamic data space and navigation space. Property actions can modify the active properties, and in some cases other related properties as well. Navigation actions can result in propagating the active Neural Message to other Neural Objects, spawning the normally dormant component of the active Neural Message to a specific Neural Object, selectively spawning new Messages associated with other properties, or aborting the propagation of the active Neural Message and all of its derivatives.

Recommended AI policy actions for energy consumption and efficiency

- **De-couple data processing from data transfers** to make the two completely independent from each other. This will allow plug-and-play addition or removal of processors and other components without having to modify existing hardware or software. It will also enable updates to the data communications infrastructure without disturbing data processing components
- **Use packet-based Multi-Dimensional Data Transfer Fabric** instead of buses to autonomously and simultaneously transfer thousands of Neural Messages between connected Neural Objects. Three-Dimensional or Four-Dimensional Fabrics stretch communication resources into new routing dimensions to increase overall data bandwidth and shorten effective communication distances between data senders and receivers (with each additional dimension used). Connecting massive amounts of processing elements with legacy buses introduces control and data flow bottlenecks that make them incompatible with data flow architectures required for massive parallelism.
- **The Data Communication Fabric should be fully autonomous.** On the inside, the communications fabric should be 100% self-regulating, self-arbitrating and self-propagating. It should not require any active control inputs from data transfer ports other than responding to Halt signals when pumping data too fast into the Fabric.
- **The Data Communications Fabric should be** structured to efficiently handle both short and long data transfers.
- **Computer hardware optimized for AI should also work for General Purpose applications.** To that effect the architecture should be fully modular and configurable so that no software changes are required when adding or removing processing hardware. Likewise, no hardware changes should be required when shifting from AI to non-AI applications regardless of the size of the computer.
- **Use point-to-point links for data communications.** This substantially reduces system power consumption especially when thousands of simultaneous data transfers are taking place. Power is reduced because data senders only see a capacitive charge associated with just one data recipient, versus many data recipients when charging common bus lines, for example.

Current status of the Neural Messaging Computer prototype

- Completed design and test of all hardware building blocks for instruction processing, data processing, and data communications, including Neural Messaging
- Completed design of the programming model and its hardware and software components
- Assembled the first HDL version of the Neural Messaging Computer
- Completed design of the instruction set and development of the assembler tool
- Completed sourcing of the first software test program that exercises all design features
- Completed HDL simulation testbench that validates all design features as directed by the software test program
- Successfully validated the operation of the Neural Messaging Computer using the test software and observing results with the HDL testbench

Next plans for Neural Messaging Computer

- Transition the software from test to an actual AI example
- Expand the tool suite to include software simulator
- Build a hardware platform for software development

About the author

- Computer architect specializing in high performance data communications and processing
- Programmed FPGA-based test platforms for Texas Instruments Digital Signal Processors
- Designed, built and productized computer test platforms used to test every new Texas Instruments C2000 processor developed between years 2012 and 2022
- Awarded 4 US patents in areas of high-performance data communications